Rapport de Langages matériels et réutilisation

ETN5 - SETR

Élèves :

Nolan BUCHET

Maxence BOULEAU

Date :

Octobre 2026

Table des matières

1 Introduction **2**

2 Récupération d'un modèle de RAM **3**
2.1 Validation du fonctionnement de la RAM . . . 3
2.2 Modification du code de la RAM . . . 3
2.3 Ajout de la généricité de la RAM . . . 3
2.4 Ajout du signal de sélection . . . 4

3 Création du Bloc QuadRAM **6**
3.1 Conception et validation du décodeur d'adresses . . . 6
3.2 Instanciation de 4 blocs SyncRam . . . 6
3.3 Généralisation de l'architecture . . . 7
3.4 Validation de la QuadRAM . . . 7

4 Prise en main d'un modèle de processeur **9**
4.1 RISC-V RV32I et chargement des instructions . . . 9
4.2 Analyse des modes d'adressage et extension RV32IM . . . 10
4.2.1 Modes d'adressage . . . 10
4.2.2 Évolution vers le jeu d'instructions RV32IM . . . 11
4.3 Passage du jeu d'instructions RV32I au jeu RV32IM . . . 11

5 Intégration Système : Processeur et Mémoire **13**
5.1 Modification du processeur . . . 13
5.2 Création d'un bloc Top-Level . . . 13
5.3 Validation du système intégral . . . 14

6 Mise en œuvre matérielle et synthèse sous Vivado **16**
6.1 Modification du code existant . . . 16
6.2 Contraintes de synthèse et préservation des signaux sous Vivado . . . 16
6.3 Analyse de la synthèse et de l'implémentation . . . 16

7 Conclusion **20**

1 Introduction

Ce mini-projet porte sur la conception, l'interfaçage et la validation par simulation sous ModelSim d'un système numérique associant un processeur RISC-V à une mémoire modulaire. Il met en œuvre les principes de généricité et de réutilisation de blocs matériels à travers une architecture mixte VHDL/Verilog.

La démarche débute par l'étude d'une mémoire RAM synchrone en VHDL, modifiée pour être paramétrable et dotée d'un signal de sélection (*Chip Select*). Quatre instances de cette mémoire sont ensuite regroupées au sein d'une entité `QuadRAM`, pilotée par un décodeur d'adresses dédié. En parallèle, le processeur RISC-V RV32I décrit en Verilog est enrichi pour prendre en charge l'extension arithmétique M (multiplication et division entières) et adapté afin de déporter le stockage des données vers la mémoire externe. Enfin, l'ensemble est interconnecté au sein d'un bloc *top-level* et validé par l'exécution de programmes de test sous ModelSim, confirmant la bonne exécution des calculs et la fiabilité des accès mémoire.

2 Récupération d'un modèle de RAM

2.1 Validation du fonctionnement de la RAM

Nous récupérons le modèle `ramchip.vhd` et son testbench afin de pouvoir observer et analyser son comportement sur ModelSim.

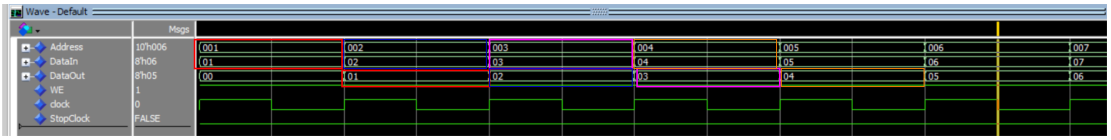


FIGURE 1 – Chronogramme de la RAM

Dans le chronogramme ci-dessus, nous observons que notre simulation représente correctement le comportement d'une RAM.

Le bon fonctionnement se valide en activant `we`. À chaque front montant de `clock`, nous définissons l'adresse `address` de la donnée à écrire, puis nous plaçons cette donnée sur `datain`.

— À 20 ns : écriture de 01 à l'adresse 001 ; `dataout` restitue 01
— À 30 ns : écriture de 02 à l'adresse 002 ; `dataout` restitue 02
— À 40 ns : écriture de 03 à l'adresse 003 ; `dataout` restitue 03, et ainsi de suite

Un effet de décalage apparait, la valeur affichée correspond toujours à la donnée appliquée au cycle précédent. Exemple entre 20 ns et 30 ns, `address` vaut 001, `datain` vaut 01 et `dataout` restitue 00. Cet effet est normal car notre RAM est synchrone et ne restitue donc la donnée qu'au cycle suivant.

2.2 Modification du code de la RAM

Dans un second temps, nous mettons à jour le code en remplaçant les `std_logic_vector` par des `unsigned` afin de pouvoir effectuer des opérations arithmétiques sur les adresses. Le fonctionnement reste identique à celui de la RAM initiale.

2.3 Ajout de la généricité de la RAM

Pour rendre notre RAM plus générique, nous ajoutons des paramètres de taille de données et d'adresses. Nous ajoutons donc les généricités `DATA_LENGHT` et `ADDR_LENGHT` qui permettent de définir la taille de la RAM.

```
1 entity sync_ram is
2   generic (
3     ADDR_LENGHT : integer := 10;
4     DATA_LENGHT : integer := 8
```

```
  );
  port (
    clock   : in  std_logic;
    we      : in  std_logic;
    address : in  unsigned((ADDR_LENGHT - 1) downto 0);
    datain  : in  unsigned((DATA_LENGHT - 1) downto 0);
    dataout : out unsigned((DATA_LENGHT - 1) downto 0)
  );
end entity sync_ram;
```

Listing 1 – Nouvelle entité de la RAM

2.4 Ajout du signal de sélection

L'ajout du signal `cs` permet de contrôler l'accès à la RAM et de placer sa sortie en haute impédance (`'Z'`) lorsqu'elle n'est pas sélectionnée, en vue de la création de la `quadram`.

```
entity sync_ram is
  generic(DataLenght : INTEGER := 8; AddrLenght : INTEGER := 8);
  port (
    cs    : in std_logic;
    clock : in std_logic;
    ...
  );
end entity;

architecture RTL of sync_ram is
  ...
begin
  process(clock) is
  begin
    if rising_edge(clock) and cs = '1' then
      -- Ecriture synchronisee
    end if;
  end process;

  dataout <= ram(to_integer(unsigned(read_address))) when cs = '1
     ' else (others => 'Z');
end architecture;
```

Listing 2 – Ajout du signal cs

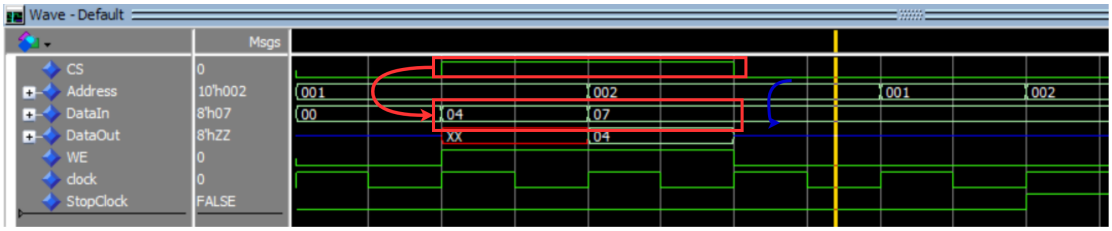


FIGURE 2 – Chronogramme de la RAM avec le signal `cs`

La Figure 2 présente le chronogramme de la RAM après l'ajout du signal de sélection `cs`. Lorsque `cs` vaut `'1'`, la RAM est sélectionnée : l'écriture est autorisée sur un front montant de `clock` si `we` est également actif, et la sortie `dataout` restitue la donnée mémorisée à l'adresse demandée. Lorsque `cs` vaut `'0'`, la mémoire n'est pas sélectionnée et sa sortie est placée en haute impédance (`'Z'`), afin de ne pas perturber un éventuel bus partagé.

Le chronogramme illustre le comportement synchrone de l'écriture : les données de `datain` sont enregistrées sur un front montant de `clock`, lorsque `cs` et `we` sont actifs. La sortie `dataout` est, quant à elle, conditionnée par `cs`. L'ajout de ce signal complète donc le fonctionnement de la RAM en permettant de contrôler son accès.

3 Création du Bloc QuadRAM

3.1 Conception et validation du décodeur d'adresses

Un décodeur d'adresses générique a été conçu pour piloter les signaux `cs` des blocs RAM à partir des bits de poids fort de l'adresse. Son fonctionnement a été validé par simulation (Figure 3).

```
entity decoder is
  generic(SEL_LENGHT : INTEGER := 2);
  port (
    address : in  unsigned((SEL_LENGHT - 1) downto 0);
    cs      : out unsigned(((2**SEL_LENGHT) - 1) downto 0)
  );
end entity;

architecture RTL of decoder is
begin
  process(address) is
  begin
    cs <= (others => '0');
    cs(to_integer(address)) <= '1';
  end process;
end architecture;
```

Listing 3 – Décodeur d'adresses générique



FIGURE 3 – Chronogramme du décodeur d'adresses

Ainsi, sur le chronogramme de la Figure 3, le vecteur de sortie `cs` évolue en fonction de la valeur du signal d'entrée `address`. Pour une adresse codée sur deux bits, quatre combinaisons sont possibles : `00`, `01`, `10` et `11`. Chacune active un bit de `cs`, tandis que les autres restent à zéro. Ce comportement permet de sélectionner le bloc mémoire correspondant à l'adresse fournie. La simulation confirme le bon fonctionnement du décodeur et son utilisation dans l'architecture de la `quadram`.

3.2 Instanciation de 4 blocs SyncRam

L'entité `QuadRAM` intègre un décodeur d'adresses et une boucle `for..generate` pour instancier les blocs mémoire. Le décodeur analyse les bits de poids fort de l'adresse pour piloter individuellement les signaux `cs` de chaque RAM, tandis que les autres signaux

(horloge, écriture et données) sont mutualisés.

```
DECODER : entity work.decoder(RTL)
generic map (
    SEL_LENGHT => 2
)
port map (
    cs      => cs_array,
    address => address(9 downto 8)
);

GenRam : for i in 0 to (2**2) - 1 generate
    Ram : entity work.sync_ram(RTL)
    generic map (
        DataLenght => 8,
        AddrLenght => 8
    )
    port map (
        cs      => cs_array(i),
        clock   => clock,
        we      => we,
        address => address(7 downto 0),
        datain  => datain,
        dataout => dataout
    );
end generate;
```

Listing 4 – QuadRAM

3.3 Généralisation de l'architecture

Pour généraliser l'architecture de la RAM, nous avons ajouté des généricités pour la taille des données et des adresses. Cela permet de définir la taille de la RAM en fonction des besoins spécifiques du projet.

```
  generic(ADDR_SELECT : INTEGER := 2;
        ADDR_LENGHT : INTEGER := 10;
        DATA_LENGHT : INTEGER := 8
  );
```

Listing 5 – QuadRAM Généralisée

3.4 Validation de la QuadRAM

Afin de valider le fonctionnement de la QuadRAM, nous avons réalisé une simulation en utilisant un test bench. Le chronogramme obtenu (Figure 4) montre que la QuadRAM fonctionne correctement, avec les signaux `cs` activés pour chaque bloc RAM en fonction des bits de poids fort de l'adresse.

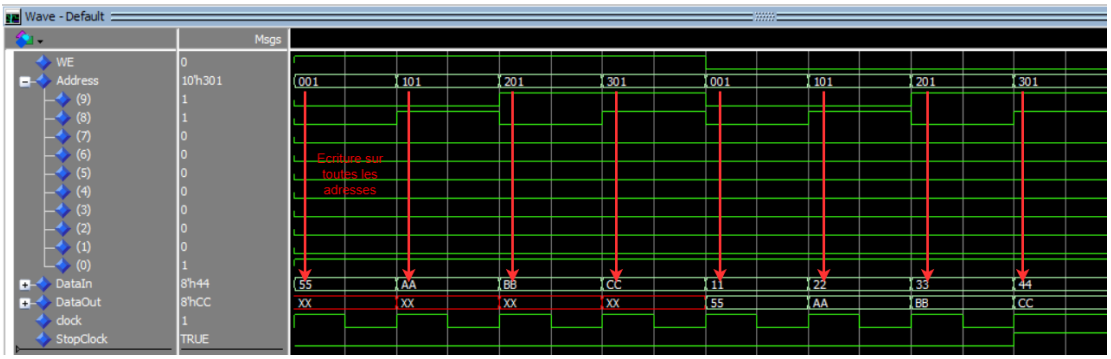


FIGURE 4 – Chronogramme de la QuadRAM

La Figure 4 présente le chronogramme de validation de la `quadram`, composée de quatre blocs RAM pilotés par un décodeur d'adresses. Les 2 bits de poids fort de `address` activent le bit correspondant de `cs_array`, tandis que les 8 bits de poids faible définissent l'adresse interne de la banque sélectionnée. Ainsi, une seule RAM est activée parmi les quatre instances. Cette simulation vérifie le décodage et la sélection des mémoires, nécessaires à l'utilisation de la `quadram` comme mémoire modulaire.

4 Prise en main d'un modèle de processeur

4.1 RISC-V RV32I et chargement des instructions

Dans un premier temps, nous avons étudié le jeu d'instructions RISC-V RV32I et chargé les instructions dans la mémoire du processeur. Un *testbench* a été créé afin de simuler le processeur et de vérifier le bon fonctionnement des instructions.

Le processeur n'exécute pas directement les instructions assembleur lisibles par un humain, comme `addi` ou `add` : chaque instruction doit être encodée en code machine selon le format défini par le jeu d'instructions. Ces mots binaires de 32 bits sont représentés en hexadécimal. Chaque groupe de huit chiffres hexadécimaux correspond ainsi à une instruction.

L'exemple ci-dessous illustre le chargement d'un programme de test :

```
// instructions at address 0x0
@0
00200093 // addi x1, x0, 2
00400113 // addi x2, x0, 4
00110133 // add x2, x2, x1
001150b3 // srl x1, x2, x1
0420a023 // sw x2, 64(x1)
00100073 // ebreak
```

Listing 6 – Exemple de chargement d'instructions

Les chronogrammes ci-dessous mettent en évidence l'exécution séquentielle de ces instructions, avec le comportement des registres (Figure 5) et l'écriture en mémoire (Figure 6) :

- **Étape 1** : le registre `x1` est initialisé à 2, puis `x2` à 4.
- **Étape 2** : leur addition est stockée dans `x2`.
- **Étape 3** : un décalage logique à droite de `x2` par `x1` est affecté à `x1`.
- **Étape 4** : la valeur de `x2` est sauvegardée en mémoire à l'adresse calculée à partir de `x1` et de l'offset 64.

Les figures suivantes détaillent respectivement le comportement interne des registres (Figure 5) et l'écriture effective des données dans l'espace mémoire.

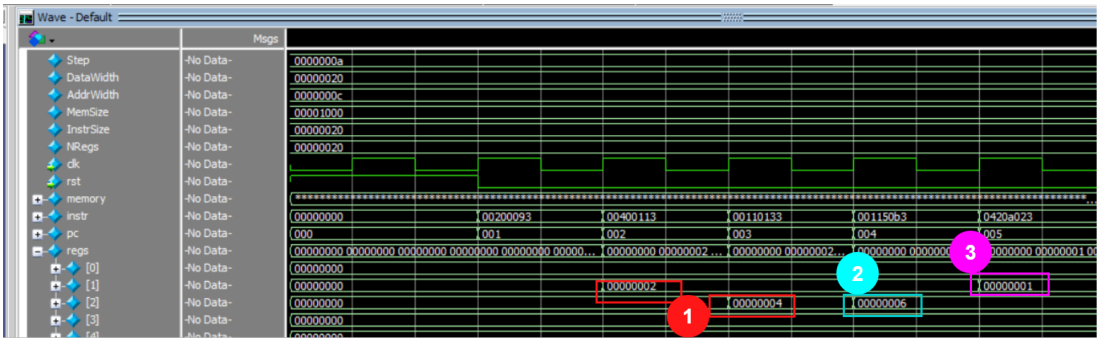


FIGURE 5 – Détail des opérations dans les registres

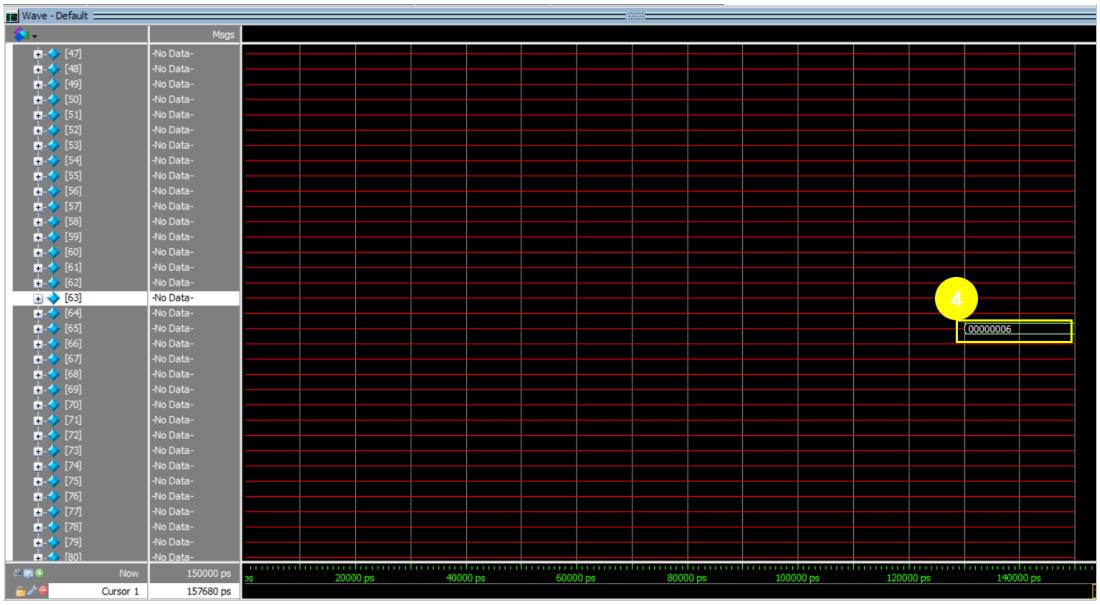


FIGURE 6 – Validation du stockage de la valeur en mémoire

4.2 Analyse des modes d'adressage et extension RV32IM

4.2.1 Modes d'adressage

L'architecture RISC-V manipule différents modes d'adressage pour accéder aux données :

— **Adressage immédiat :** La valeur de la donnée est directement encodée au sein même de l'instruction (ex : `addi x1, x0, 2`).

— **Adressage direct :** L'adresse de la mémoire ciblée est explicitement fournie dans l'instruction.

— **Adressage indirect (basé sur un registre) :** L'adresse mémoire effective est contenue dans un registre, souvent combinée avec un décalage constant (*offset*), comme dans l'instruction de stockage `sw x2, 64(x1)`.

4.2.2 Évolution vers le jeu d'instructions RV32IM

Le processeur fourni gère nativement le jeu d'instructions de base RV32I. Pour le rendre compatible avec l'extension **M** (*Multiplication and Division*), les modifications matérielles suivantes sont requises :

- Intégration d'unités de calcul arithmétique dédiées dans l'ALU pour la multiplication et la division entières (instructions `mul`, `div`).
- Décodage des nouveaux *opcodes* et *funct3/funct7* spécifiques à l'extension M dans l'unité de contrôle.
- Adaptation du programme de test (‘instr.mem‘) pour valider ces nouvelles opérations arithmétiques.

4.3 Passage du jeu d'instructions RV32I au jeu RV32IM

Voici les modifications apportées au code Verilog du processeur pour supporter l'extension M :

```
// Add functions RV32IM
‘define MUL    {3’b000, 7’b0000001}
‘define DIV    {3’b100, 7’b0000001}

// Addictionnal function
‘MUL: regs[‘RD] <= regs[‘RS1] * regs[‘RS2];
‘DIV: regs[‘RD] <= regs[‘RS1] / regs[‘RS2];
```

Listing 7 – Ajout de l'extension M dans le processeur

Nous modifions également le fichier ‘instr.mem‘ pour inclure des instructions :

- `div` : x2 = x2 / x1 (x2 = 4 / 2)
- `mul` : x2 = x2 * x1 (x2 = 2 * 4)

```
// div
00200093 // addi x1, x0, 2
00400113 // addi x2, x0, 4
02114133 // div x2, x2, x1

// mul
00200093 // addi x1, x0, 2
00400113 // addi x2, x0, 4
02110133 // mul x2, x2, x1
```

Listing 8 – Instructions pour l'extension M

Voici le chronogramme résultant de l'exécution de ces instructions (Figure 7) :

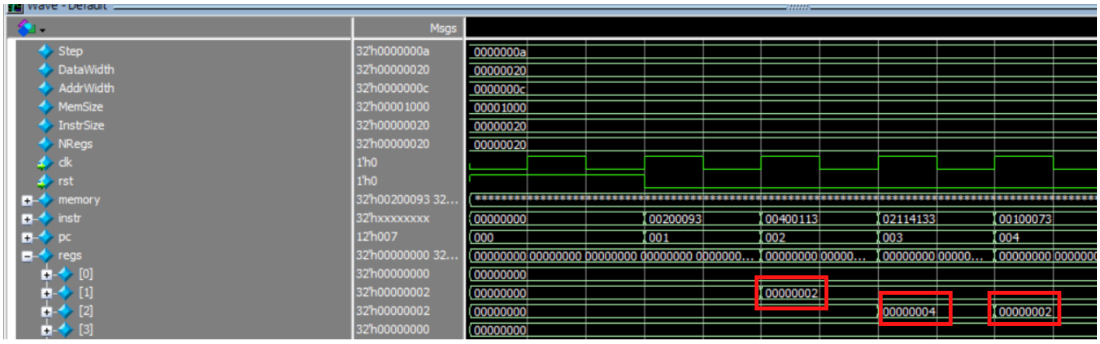


FIGURE 7 – Chronogramme du processeur pour division

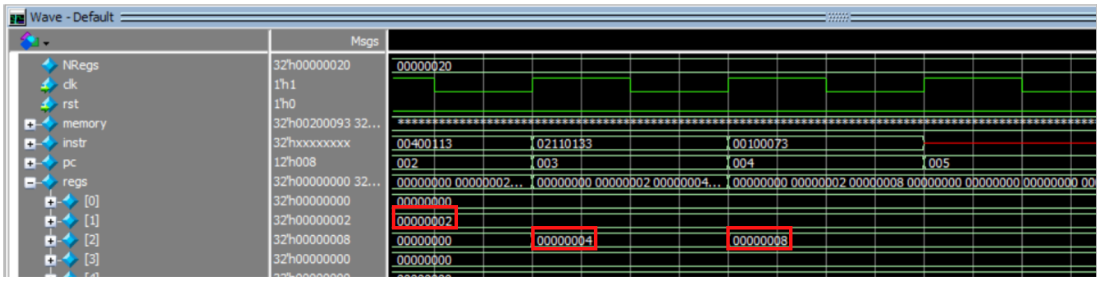


FIGURE 8 – Chronogramme du processeur pour multiplication

5 Intégration Système : Processeur et Mémoire

5.1 Modification du processeur

Le processeur a été modifié pour déporter son stockage de données vers la mémoire externe `quadram`, en intégrant les bus `mem_addr`, `mem_wdata` et `mem_rdata`, ainsi que les signaux de contrôle `mem_we`, `clk` et `rst`.

```
module system (
  input wire clk, rst,
  output reg  [11:0] mem_addr,
  output reg  [31:0] mem_wdata,
  input  wire [31:0] mem_rdata,
  output reg         mem_we
);
```

Listing 9 – Interface externe du processeur

5.2 Création d'un bloc Top-Level

Un composant *Top-Level* rédigé en VHDL relie le processeur (en Verilog) et le bloc `QuadRAM`, en assurant la conversion des types entre les signaux standards et les vecteurs non signés.

```
entity system_top is
    Port (
        clk : in STD_LOGIC;
        rst : in STD_LOGIC
    );
end entity system_top;

architecture RTL of system_top is

    signal mem_addr  : std_logic_vector(11 downto 0);
    signal mem_wdata : std_logic_vector(31 downto 0);
    signal mem_rdata : std_logic_vector(31 downto 0);
    signal mem_we    : std_logic;

    signal quadram_addr  : unsigned(11 downto 0);
    signal quadram_wdata : unsigned(31 downto 0);
    signal quadram_rdata : unsigned(31 downto 0);

begin

    quadram_addr  <= unsigned(mem_addr);
    quadram_wdata <= unsigned(mem_wdata);
    mem_rdata     <= std_logic_vector(quadram_rdata);

    processor_inst : entity work.system
        port map (
            clk       => clk,
            rst       => rst,
```

```
29              mem_addr  => mem_addr,
30              mem_wdata => mem_wdata,
31              mem_rdata => mem_rdata,
32              mem_we    => mem_we
33          );
34
35      quadram_inst : entity work.quadram
36          generic map (
37              ADDR_SELECT => 2,
38              ADDR_LENGHT => 12,
39              DATA_LENGHT => 32
40          )
41          port map (
42              clock    => clk,
43              we       => mem_we,
44              address  => quadram_addr,
45              datain   => quadram_wdata,
46              dataout  => quadram_rdata
47          );
48
49  end architecture RTL;
```

Listing 10 – Extrait du Top-Level système

5.3 Validation du système intégral

Dans le même esprit que pour la validation du modèle processeur, nous validons notre système intégral en simulant l'exécution d'un programme de test. Le chronogramme ci-dessous (Figure 9) illustre le fonctionnement du processeur en interaction avec la mémoire `quadram`.

- **Étape 1** : le processeur additionne `x1` = 2 et `x2` = 4, puis écrit le résultat 6 à l'adresse mémoire 0.
- **Étape 2** : il multiplie `x1` = 2 par `x2` = 4, puis écrit le résultat 8 à l'adresse mémoire 1.
- **Étape 3** : il divise `x2` = 4 par `x1` = 2, puis écrit le résultat 2 à l'adresse mémoire 2.
- **Étape 4** : il lit le contenu de l'adresse mémoire 0 et charge la valeur 6 dans `x2`.

Voici le chronogramme obtenu (Figure 9) :

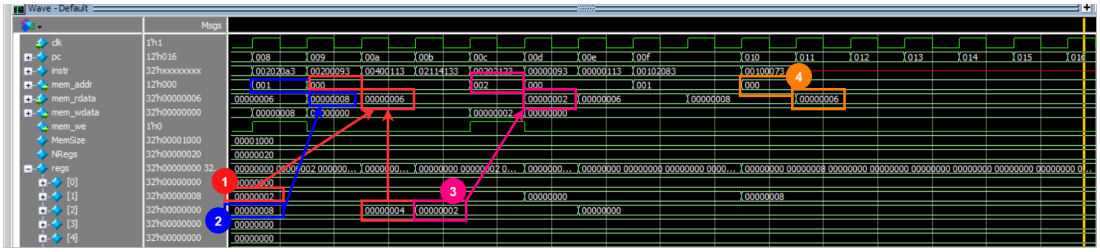


FIGURE 9 – Opérations arithmétiques et échanges mémoire du processeur avec la `quadram`

Les repères colorés du chronogramme suivent ces quatre étapes : le rouge identifie l'addition et l'écriture de 6 à l'adresse 0 ; le bleu, la multiplication et l'écriture de 8 à l'adresse 1 ; le rose, la division et l'écriture de 2 à l'adresse 2 ; enfin, l'orange correspond à la lecture de l'adresse 0 et au chargement de la valeur 6 dans `x2`.

6 Mise en œuvre matérielle et synthèse sous Vivado

Pour rendre le système complet synthétisable sur un FPGA Xilinx avec Vivado, deux adaptations principales sont nécessaires : modifier le modèle de mémoire pour utiliser les blocs mémoire physiques du FPGA et rendre le processeur réellement synchrone sur le signal d'horloge. Le code doit également respecter les contraintes de synthèse et les ressources disponibles sur le composant cible.

6.1 Modification du code existant

Dans notre projet, le processeur (RISC-V) est écrit en Verilog tandis que la mémoire et le niveau supérieur (*Top-Level*) sont en VHDL. Pour garantir la synthétisabilité de l'ensemble mixte :

— Un nettoyage des constructions de simulation non synthétisables a été réalisé (comme la suppression ou l'isolation des instructions de debug de type `$display` dans le processeur Verilog).

— Les sensibilités d'horloge et de réinitialisation ont été standardisées pour correspondre aux attentes des outils de synthèse FPGA.

6.2 Contraintes de synthèse et préservation des signaux sous Vivado

Durant l'étape de synthèse, l'outil analyse le design et applique des optimisations agressives (suppression de la logique jugée redondante ou non connectée aux broches physiques de sortie). Étant donné que notre système fonctionne de manière autonome en interne sans relier tous les signaux mémoire à des broches externes du FPGA, Vivado pourrait être tenté d'optimiser et de supprimer les bus d'interconnexion.

Pour empêcher cette suppression indésirable et conserver une visibilité complète sur la netlist, nous avons appliqué l'attribut de synthèse **`DONT_TOUCH`** sur les signaux intermédiaires du top-level (Listing 11).

```
attribute DONT_TOUCH : string;
attribute DONT_TOUCH of mem_addr      : signal is "true";
attribute DONT_TOUCH of mem_wdata     : signal is "true";
attribute DONT_TOUCH of mem_rdata     : signal is "true";
attribute DONT_TOUCH of mem_we        : signal is "true";
attribute DONT_TOUCH of quadram_addr  : signal is "true";
attribute DONT_TOUCH of quadram_wdata : signal is "true";
attribute DONT_TOUCH of quadram_rdata : signal is "true";
```

Listing 11 – Extrait du Top-Level système avec application de l'attribut DONT_TOUCH sous Vivado

6.3 Analyse de la synthèse et de l'implémentation

Ci-dessous, les différentes vues issues de l'outil Vivado illustrant les étapes de synthèse et d'implémentation :

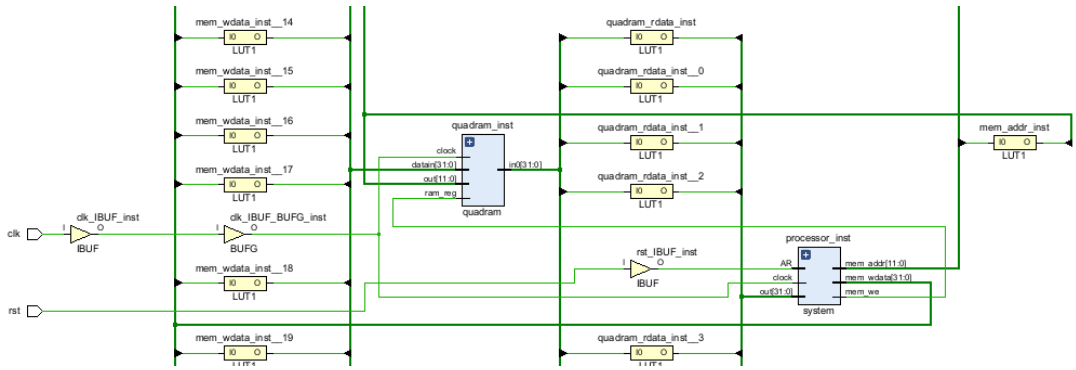
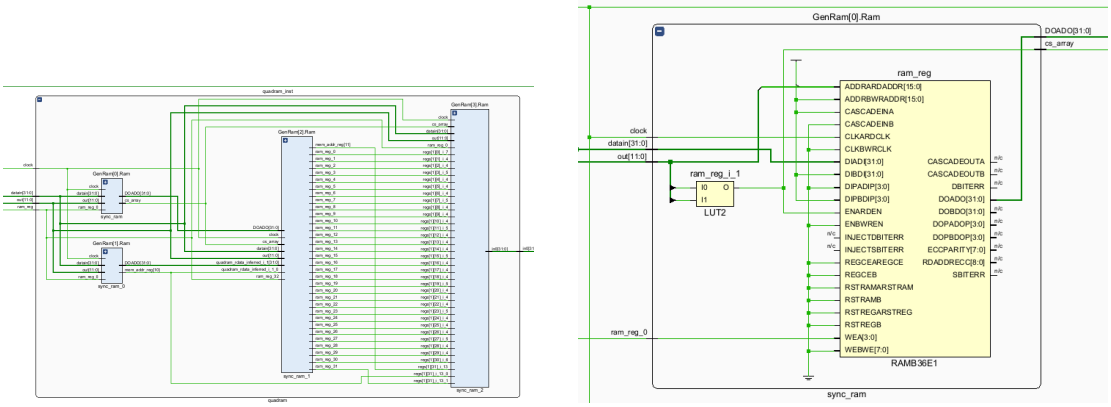


FIGURE 10 – Schéma global de synthèse

Dans cette première capture, nous observons le schéma global de synthèse du design, où les blocs principaux (processeur et QuadRAM) sont interconnectés.



(a) Vue du module QuadRAM (b) Génération du design (`gen0`)

FIGURE 11 – Vues schématiques de la synthèse sous Vivado

En zoomant sur la QuadRAM, nous pouvons observer les quatre blocs de mémoire RAM. La génération du design (`gen0`) met en évidence l'utilisation d'un bloc de mémoire interne au FPGA (RAMB36E1), optimisant ainsi l'utilisation des ressources logiques.

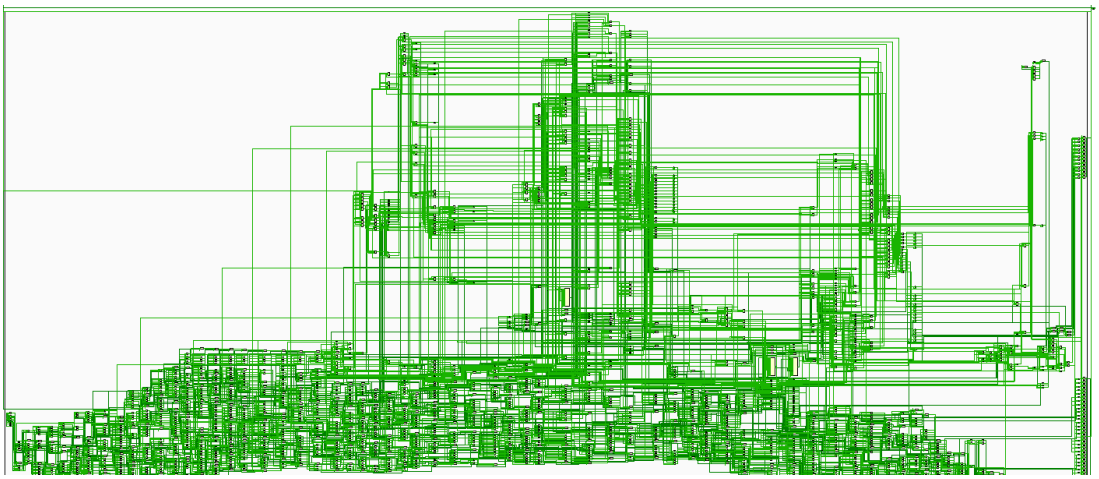


FIGURE 12 – Vue du processeur RISC-V

Le processeur RISC-V s'avère être le module le plus complexe à analyser en détail. Étant capable d'exécuter différentes opérations, ce schéma regroupe logiquement les différents sous-modules : l'unité arithmétique et logique (ALU), le décodeur d'instructions, le compteur ordinal (programme) et les registres internes.

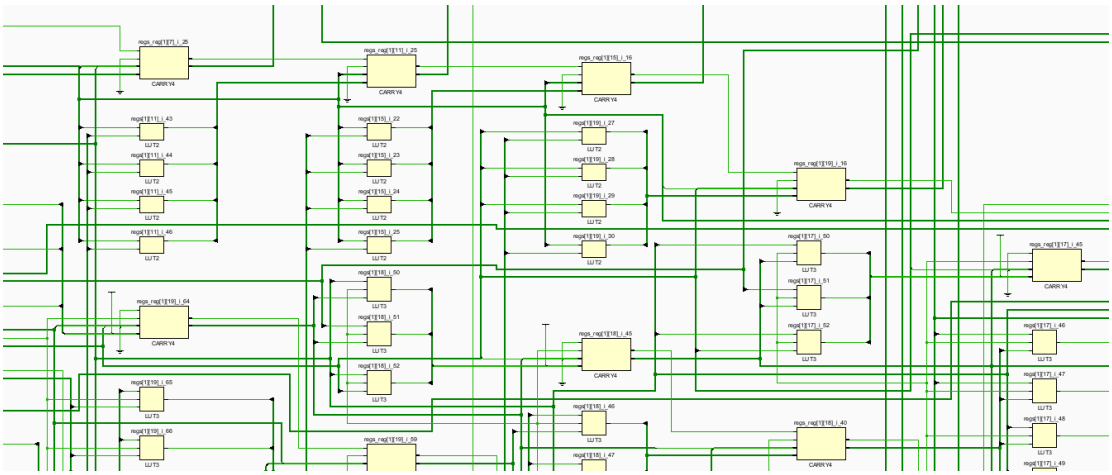


FIGURE 13 – Zoom sur le processeur

La figure ci-dessus présente un zoom sur une région spécifique du processeur, permettant d'observer la disposition des différents sous-modules internes.

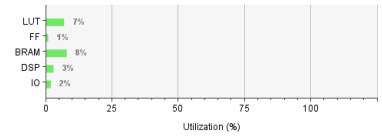
Grâce au rapport d'utilisation des ressources, nous constatons que le design utilise seulement 7 % des LUTs et 1 % des FFs disponibles sur le FPGA. Cela témoigne de la légèreté de l'architecture, laissant une marge importante pour de futures extensions.

Au total, le design mobilise 1 513 LUTs, réparties entre 66 pour la QuadRAM et 1 399 pour le processeur. Cela confirme que ce dernier représente la majeure partie de la

Name	Slice LUTs (20800)	Slice Registers (41600)	F7 Muxes (16300)	F8 Muxes (8150)	Slice (8150)	LUT as Logic (20800)	Block RAM Tile (50)	DSPs (90)	Bonded IOB (106)	BUFGCTRL (32)
N system_top	1513	154	37	13	462	1513	4	3	2	1
processor_inst (system)	1399	154	37	13	440	1399	0	3	0	0
quadram_inst (quadram)	66	0	0	0	27	66	4	0	0	0

(a) Rapport d'utilisation des ressources (1)

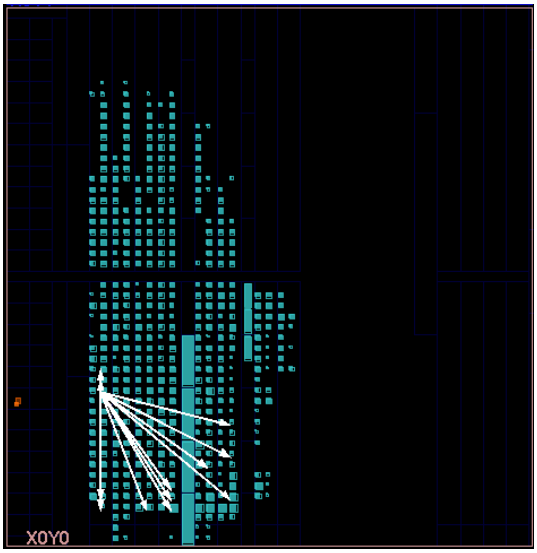
Resource	Utilization	Available	Utilization %
LUT	1513	20800	7.27
FF	154	41600	0.37
BRAM	4	50	8.00
DSP	3	90	3.33
IO	2	106	1.89



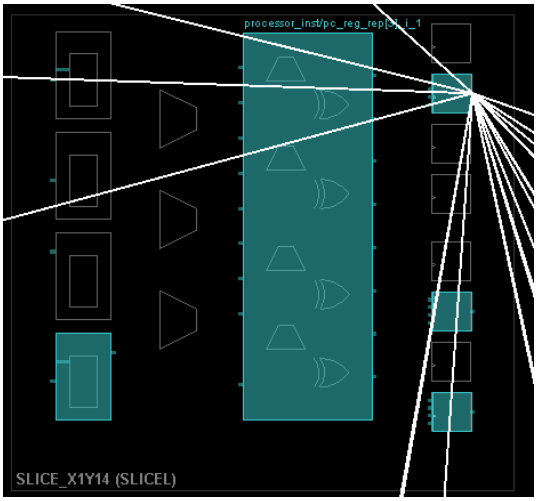
(b) Rapport d'utilisation des ressources (2)

FIGURE 14 – Rapports d'utilisation des ressources du design sous Vivado

complexité logique du système, ce qui est cohérent avec la gestion des instructions et des accès mémoire.



(a) Vue de l'implémentation physique



(b) Zoom sur le routage

FIGURE 15 – Vues de l'implémentation et du placement-routage sur le FPGA

Enfin, l'implémentation physique du design sur le FPGA révèle la disposition spatiale des différents blocs. Le zoom sur le routage illustre les interconnexions réalisées par l'outil entre les blocs logiques. Nous pouvons notamment observer le signal `pc_reg_rep_0`, correspondant à la sortie du registre de programme du processeur et implémenté par une bascule D (*D Flip-Flop*).

7 Conclusion

Ce projet nous a permis de concevoir et de valider un système associant un processeur RISC-V à une mémoire externe modulaire. La RAM générique, la `QuadRAM` et le processeur étendu aux opérations de multiplication et de division ont été intégrés puis testés ensemble sous ModelSim. Les simulations confirment le bon fonctionnement des opérations arithmétiques et des échanges mémoire.

La synthèse et l'implémentation sous Vivado ont ensuite permis de vérifier la compatibilité du système avec une cible FPGA. La `QuadRAM` est notamment mappée sur un bloc mémoire interne, tandis que l'utilisation des ressources reste limitée à environ 7 % des LUTs et 1 % des bascules.